


```

    }
    if ((comp.kind === 'subtable' || comp.kind === 'table') && comp.permColumns) {
      const hide = comp.permColumns[role] || [];
      if (hide.length && comp.subColumns) comp.subColumns = comp.subColumns.filter(c => !hide.includes(c));
      if (hide.length && comp.dataBinding?.columns) comp.dataBinding.columns = comp.dataBinding.columns.filter(c => !hide.includes(c))
    }
    // ██████████ $6██████████████████████ maskMetricsByPerm ████████
    if (comp.kind === 'kpi-strip' && comp.permMetrics) {
      const hiddenKeys = [];
      for (const [permKey, metricKeys] of Object.entries(comp.permMetrics)) {
        const vis = FIELD_PERMS[permKey]?.[role] || 'visible';
        if (vis === 'hidden') hiddenKeys.push(...(metricKeys || []));
      }
      if (hiddenKeys.length) comp.permHiddenKeys = hiddenKeys;
    }
    if (comp.kind === 'pipeline' && comp.permStages) {
      const hiddenStages = [];
      for (const [permKey, stageKeys] of Object.entries(comp.permStages)) {
        const vis = FIELD_PERMS[permKey]?.[role] || 'visible';
        if (vis === 'hidden') hiddenStages.push...(stageKeys || []);
      }
      if (hiddenStages.length) comp.permStagesHiddenKeys = hiddenStages;
    }
    if (comp.kind === 'progress-card' && comp.permKey) {
      const vis = FIELD_PERMS[comp.permKey]?.[role] || 'visible';
      if (vis === 'hidden') comp.permHidden = true;
    }
  }
  return s;
}

```

```
// ##### applyFieldPerms ### schema #####
// kpi-strip → ##### state:'hidden' pipeline → ##### amount ### null progress-card → ## hidden
export function maskMetricsByPerm(schema, data) {
  const out = structuredClone(data);
  for (const comp of schema.components || []) {
    const group = out.components?.[comp.kind];
    if (!group || typeof group !== 'object') continue;
    const key = comp.title;
    if (key == null || !Object.prototype.hasOwnProperty.call(group, key)) continue;
    const datum = group[key];
    if (comp.kind === 'kpi-strip' && comp.permHiddenKeys?.length && Array.isArray(datum.items)) {
      datum.items = datum.items.map((it) => (comp.permHiddenKeys.includes(it.key)
        ? { key: it.key, label: it.label, value: null, unit: '', state: 'hidden', hint: '' }
        : it));
    }
    if (comp.kind === 'pipeline' && comp.permStagesHiddenKeys?.length && Array.isArray(datum.stages)) {
      const hidden = new Set(comp.permStagesHiddenKeys);
      datum.stages = datum.stages.map((s) => (hidden.has(s.key) ? { ...s, amount: null } : s));
      datum.permHiddenStages = comp.permStagesHiddenKeys;
    }
  }
}
```

[illegible]


```
// #####noOverdueScore##### taskOverdue ■ null■ due ■#####
// #####100#####2026-09-03 #####
let healthScore = null;
if (contractAmt !== null || interactions > 0 || decTotal > 0) {
  const rateScore = clamp100(payRate ?? 0);
  const actScore = clamp100((interactions30d / 5) * 100);
  const noOverdueScore = taskOverdue === null ? null : (taskOverdue === 0 ? 100 : clamp100(100 - taskOverdue * 25));
  if (decTotal > 0) {
    const comps = [['rate', 0.3, rateScore], ['act', 0.3, actScore], ['exc', 0.2, 100 - (excRate ?? 0)]];
    if (noOverdueScore !== null) comps.push(['overdue', 0.2, noOverdueScore]);
    const wsum = comps.reduce((s, c) => s + c[1], 0);
    healthScore = Math.round(comps.reduce((s, c) => s + c[1] * c[2], 0) / wsum);
  } else {
    const comps = [['rate', 0.45, rateScore], ['act', 0.45, actScore]];
    if (noOverdueScore !== null) comps.push(['overdue', 0.1, noOverdueScore]);
    const wsum = comps.reduce((s, c) => s + c[1], 0);
    healthScore = Math.round(comps.reduce((s, c) => s + c[1] * c[2], 0) / wsum);
  }
}
const healthState = healthScore === null ? 'neutral' : (healthScore >= 70 ? 'good' : healthScore >= 40 ? 'warn' : 'bad');

return {
  money: { contractAmt, paidAmt, unpaidAmt, payRate },
  pipeline: { stages, conversions },
  activity: { interactions, interactions30d, taskTotal, taskOverdue, lastFollowDays },
  decision: { decTotal, decExc, excRate, healthScore, healthState },
};
};
}

// — #####handler ##### accountId #####scope #####—
export async function loadRelatedParticles(accountId, dealIds = [], tenantId = 'system') {
  const types = ['CRM_CONTACT', 'CRM_DEAL', 'CRM_QUOTATION', 'CRM_CONTRACT', 'CRM_ORDER', 'CRM_PAYMENT_PLAN', 'CRM_PAYMENT_RECORD', 'CRM_PAYMENT_RECORD'];
  const byType = {};
  for (const t of types) {
    const rows = await query(
      `SELECT id, type, slug, title, payload, state FROM crm.particles
      WHERE type=$1 AND tenant_id=$4
      AND (payload->>'account_id'=$2 OR payload->>'deal_id' = ANY($3::text[]))`,
      [t, accountId, dealIds, tenantId]
    );
    byType[t] = rows.rows;
  }
  return byType;
}

// #####events / tasks / decision / memory_log■
// T8(BG-01b)##### context/timelineSource.retrieveTimeline#####
// ##### COALESCE(decided_at, created_at)■BG-08##### SQL
export async function loadTimelineSources(accountId, dealIds = [], tenantId = 'system') {
  return retrieveTimeline({ accountId, dealIds, tenantId });
}
```



```

    Number.isFinite(conf) ? `${Math.round(conf * 100)}%` : '-',
    tier: p.business_tier || '-',
    actual: [p.state, p.disposition].filter(Boolean).join(' / ') || '-',
  });
});
}

// ██████████ precedent ██████████referenced_precedents █
// [{precedent_id, similarity}] ██████████ renderTable █ escapeHtml(obj)→'[object Object]'.█
// ██████████ + ██████████ ██████████
function formatPrecedents(precs) {
  if (!Array.isArray(precs) || precs.length === 0) return '█';
  return precs.map((p) => {
    const pid = typeof p === 'string' ? p : (p?.precedent_id || '');
    const sim = p && p.similarity != null ? Math.round(Number(p.similarity) * 100) : null;
    const parts = [];
    if (pid) parts.push(pid.slice(0, 8));
    if (sim != null) parts.push(`${sim}%`);
    return parts.join('.') || '-';
  }).join('█');
}

export async function loadDecisions(accountId, dealIds = []) {
  // ███ 2026-09-02██ timelineSource.js:78 ██████████involved_entities ████**███
  // █ ` @> '{"account_id":...}':jsonb` ██████ false → ██████████
  // ██████████ account_id ███ ██████████ id ██████████jsonb_typeof ██████████
  // 2026-09-03 ██████████ CRM_DEAL████ CRM_ACCOUNT ███ involved_entities██
  // ███ account_id ██████████ dealIds ██████████ DEAL ██████████
  const deals = Array.isArray(dealIds) ? dealIds : [];
  const r = await query(
    `SELECT d.decision_id::text AS id, d.scenario_id, d.disposition, d.state, d.rationale,
      COALESCE(d.referenced_precedents::text, ['']) AS precedents
    FROM crm.decision d
    WHERE d.involved_entities @> $1::jsonb
      OR (jsonb_typeof(d.involved_entities) = 'array'
        AND EXISTS (SELECT 1 FROM jsonb_array_elements(d.involved_entities) e
          WHERE e->>'id' = $2
            OR e->>'id' = ANY($3::text[])))
    -- 2026-09-03 ███ fallback██████(routes.js:502)██ involved_entities ███ id ███ null██
    -- ███ e->>'id' ███ null ██████████ → ███/███/KPI ██████████ 3 █ LEAD_FOLLOW_UP █ null██
    -- ███ trigger_context ██████████account_id ██████████ → ███+██████████
    OR d.trigger_context->>'account_id' = $2
    OR d.trigger_context->>'id' = ANY($3::text[])
    ORDER BY ${DECISION_TIME_BASIS} DESC LIMIT 50`,
    [JSON.stringify({ account_id: accountId }), accountId, deals]
  );
  return r.rows.map(d => ({
    // 2026-09-03 ██████/██████ decision████scenario_id████scenario_id████disposition██
    // ████████████████████ scenario██████████ disposition██████████
    // decision ██████ decision_id████scenario_id ███ scenario_id████state/precedent █████
  }));
}

```



```

previous_checksum text, -- 
created_at timestamptz DEFAULT now()
));
}

// 
// 
export async function recordAudit({ target_particle_type, source, action, actor = 'system', decision_id = null, payload = {} }) {
  const prev = (await query(
    `SELECT checksum FROM crm.audit_event ORDER BY id DESC LIMIT 1`
  )).rows[0];
  const previous_checksum = prev ? prev.checksum : null;
  const canonical = JSON.stringify(payload ?? {});
  const checksum = sha256((previous_checksum || '') + '|' + canonical);
  try {
    await queryWrite(
      `INSERT INTO crm.audit_event (target_particle_type, source, action, actor, decision_id, payload, checksum, previous_checksum)
      VALUES ($1,$2,$3,$4,$5,$6,$7,$8)` ,
      [target_particle_type, source, action, actor, decision_id, canonical, checksum, previous_checksum]
    );
    return { ok: true, id: null };
  } catch (e) {
    // 
    return { ok: false, error: e.message };
  }
}

// 
// priceCalc.js PG 
export async function recordPriceChangeAudit({ particle_id, before, after, reason, actor = 'system', decision_id = null }) {
  return recordAudit({
    target_particle_type: 'CRM_PRICE_LIST',
    source: 'price',
    action: 'price_change',
    actor,
    decision_id,
    // field P1 GET /api/particles/:id/field-history?field=list_price`
    payload: { particle_id, field: 'list_price', before, after, price_change_reason: reason || null },
  });
}

// 
export async function verifyAuditChain() {
  const rows = (await query(
    `SELECT id, payload, previous_checksum, checksum FROM crm.audit_event ORDER BY id`
  )).rows;
  let prev = null;
  for (const row of rows) {
    const expected = sha256((prev || '') + '|' + (row.payload ?? '{}'));
    if (expected !== row.checksum) return { status: 'TAMPERED', broken_at: row.id };
  }
}

```



```
//
//   ctx.bootstrap / def.autoDecision Action mint decision
// decision_id ctx params Action id
const decisionId = ctx.decision_id || params?.decision_id;
if (def.kind === 'write' && !decisionId && !ctx.bootstrap && !def.autoDecision) {
  emit('trace', 'action-write-blocked', { action: actionName, actor: ctx.actor, reason: 'no_decision_id' });
  return { ok: false, gate: 'decision_required', error: '0: decision_id' };
}
// T1 autoDecision mint decisionScenario handler mint + mint
// decisionScenario 7 handler mint action → mint
// scenario → trace
if (def.autoDecision && !decisionId && def.decisionScenario) {
  const sc = await getScenario(def.decisionScenario, ctx.tenantId);
  if (sc) {
    const d = await requireDecision(def.decisionScenario, { action: actionName, ...params, actor: ctx.actor }, inferEntities(actionName));
    ctx.decision_id = d.decision_id;
    emit('decision', `${actionName}-auto`, { decision_id: d.decision_id, scenario: def.decisionScenario });
  } else {
    emit('trace', 'auto-decision-no-scenario', { action: actionName, scenario: def.decisionScenario });
  }
}
// 1 / demo
// ctx.bootstrap / demo
if (!ctx.bootstrap && isParticleScoped(def)) {
  const role = await actorRole(ctx);
  if (role) {
    const profile = await loadProfile(role.role_tag);
    if (profile) {
      const verdict = await enforceScope(def, ctx, params, profile);
      if (!verdict.ok) {
        emit('trace', 'action-scope-blocked', { action: actionName, actor: ctx.actor, reason: verdict.reason });
        return { ok: false, gate: 'scope_violation', error: '1: ${verdict.reason || ''}' };
      }
    }
  }
}
// 1.5 RBAC $6.6 skill_registry.rbac_roles
// Action rbac_roles
// rbac_roles / bootstrap → 1
if (!ctx.bootstrap && Array.isArray(def.rbac_roles) && def.rbac_roles.length) {
  const role = await actorRole(ctx);
  if (!role || !def.rbac_roles.includes(role.role_tag)) {
    emit('trace', 'action-permission-denied', { action: actionName, actor: ctx.actor, role: role?.role_tag });
    return { ok: false, gate: 'permission_denied', error: '1.5: ${role?.role_tag || 'unidentified'} ${actionName}' };
  }
}
// 1.7 plan_entitlement $6
// Action requiresEntitlement
// → fail-closed → fail-open
// ctx.bootstrap** system resolveEntitlements
// P1-12026-09-06 ctx.tenantId || 'system'
```

[illegible]

```
// [REDACTED] bootstrap [REDACTED] approvalPassed [REDACTED] submit→approve→exec [REDACTED]
if (def.kind === 'write' && def.needsApproval && !ctx.bootstrap && !ctx.approvalPassed) {
  emit('trace', 'action-approval-blocked', { action: actionName, actor: ctx.actor, reason: 'needs_approval' });
  return { ok: false, gate: 'approval_required', error: '[REDACTED] HITL [REDACTED] approvalPassed=true' };
}
// [REDACTED] 3.5 [REDACTED] $7 [REDACTED] crm-deal-advance [REDACTED]
// [REDACTED] soft gate— [REDACTED] gap [REDACTED] /BANTCC [REDACTED]
// [REDACTED] skills/method-stage-progression/rules/gates.md [REDACTED] P1-P6 advance_gate [REDACTED]
// [REDACTED] crm-deal-advance [REDACTED] autoDecision [REDACTED] ctx.decision_id [REDACTED] from_stage [REDACTED]
// → [REDACTED] DEAL [REDACTED] payload.stage [REDACTED] P1-P6 [REDACTED] payload.stage [REDACTED]
if (def.name === 'crm-deal-advance' && !ctx.bootstrap && params?.deal_id && params?.to_stage) {
  try {
    const deal = await getParticle(params.deal_id);
    // [REDACTED] DB/[REDACTED] P [REDACTED] S [REDACTED] 3.5 [REDACTED]
    const curStage = toStageCode(deal?.payload?.stage) || deal?.payload?.stage || null;
    const toStage = toStageCode(params.to_stage) || params.to_stage || null;
    const dealPayload = deal?.payload || {};
    const v = salesStageGate({ curStage, toStage, dealPayload });
    if (!v.ok) {
      emit('trace', 'sales-stage-gate-blocked', { action: actionName, actor: ctx.actor, curStage, toStage, gaps: v.gaps });
      return { ok: false, gate: 'sales_prereq', error: `[REDACTED] 3.5 [REDACTED] ${v.gaps.join(';')}` };
    }
    if (v.warnings?.length) {
      ctx.salesWarnings = v.warnings;
      emit('trace', 'sales-stage-gate-soft', { action: actionName, actor: ctx.actor, curStage, toStage, warnings: v.warnings });
    }
  } catch { /* [REDACTED] fail-open [REDACTED] */ }
}
// [REDACTED] 3.6 [REDACTED] 2026-08-30 [REDACTED] C [REDACTED] C6 [REDACTED]
// [REDACTED] data-particle-create + CRM_DEAL + [REDACTED] lead [REDACTED]
// [REDACTED] bantcc.* [REDACTED] budget/authority/timetable [REDACTED] P3→P4 [REDACTED] ai.bantcc_completeness [REDACTED]
// [REDACTED] B [REDACTED] = BANTCC [REDACTED] B([REDACTED])/A([REDACTED])/T([REDACTED]) [REDACTED] P3→P4 [REDACTED]
// [REDACTED] salesDealPrereq [REDACTED] DB [REDACTED] dispatch [REDACTED]
if (def.name === 'data-particle-create' && !ctx.bootstrap
  && (params?.type === 'CRM_DEAL' || params?.particle_type === 'CRM_DEAL')) {
  const v = salesDealPrereq(params.payload || {});
  if (!v.ok) {
    emit('trace', 'sales-deal-prereq-blocked', { action: actionName, actor: ctx.actor, missing: v.missing });
    return { ok: false, gate: 'sales_deal_prereq', error: `[REDACTED] ${v.missing.join('/')}` };
  }
}
if (def.kind === 'write') {
  emit('trace', 'action-write-requested', {
    action: actionName, actor: ctx.actor, decision_id: ctx.decision_id || null, params,
  });
  // 10-[REDACTED] V5 [REDACTED] Action [REDACTED] fail-open [REDACTED]
  await recordAudit({
    target_particle_type: def.particleType || inferParticleTypeFromParams(params),
    source: 'action', action: `${actionName}:requested`, actor: ctx.actor || 'system',
    decision_id: decisionId, payload: { params },
  });
}
```

```
});  
}  
try {  
const data = await def.handler(params, ctx);  
if (def.kind === 'write') {  
emit('trace', 'action-write-executed', {  
action: actionName, actor: ctx.actor, ok: true, decision_id: ctx.decision_id || null,  
});  
// 10- V5 Action fail-open  
await recordAudit({  
target_particle_type: def.particleType || inferParticleTypeFromParams(params),  
source: 'action', action: `${actionName}:executed`, actor: ctx.actor || 'system',  
decision_id: decisionId, payload: { result_summary: (data && data.id) ? { id: data.id } : {} },  
});  
// 09-V6 token- Action token ctx.tokensIn/Out 0  
// fail-open -  
await recordTokens({  
actor: ctx.actor || 'system', action: actionName,  
// P0-2 2026-09-06 ctx.tokensIn/Out LLM ctx.tokenMeter  
// agentLoop metering.onUsage → 0  
tokensIn: ctx.tokensIn ?? ctx.tokenMeter?.in ?? 0,  
tokensOut: ctx.tokensOut ?? ctx.tokenMeter?.out ?? 0,  
source: 'llm', decision_id: decisionId,  
tenantId: ctx.tenantId || 'system',  
module: ctx.module || 'core',  
});  
}  
return { ok: true, data, action: actionName, confirm: def.confirm || null };  
} catch (e) {  
if (def.kind === 'write') {  
emit('trace', 'action-write-failed', { action: actionName, error: e.message });  
// 10- V5 Action fail-open  
await recordAudit({  
target_particle_type: def.particleType || inferParticleTypeFromParams(params),  
source: 'action', action: `${actionName}:failed`, actor: ctx.actor || 'system',  
decision_id: decisionId, payload: { error: e.message },  
});  
}  
return { ok: false, error: e.message, action: actionName };  
}  
},  
};  
  
// ■ 3.5 soft/hard  
// side dispatch crm-deal-advance hard BANTCC<0.6/ →  
// soft P4→P5 warnings  
// skills/method-stage-progression/methodology.json stages[ ].advance_gate  
// S1 →S2 →S3 →S4 →S5 →S6 →S7 /S8  
// v0.3 + BANTCC S2→S3 S3→S4 BANTCC  
// 3.5 src/sales/stageTaxonomy.js S_GATE_DEFS / S_ATTACHMENT_GATES check
```



```
    }  
    const missing = detail  
      ? Object.entries(detail).filter(([v]) => Number(v) < pass).map([(k)] => k)  
        : [];  
    return missing.length  
      ? `BANTCC █████<${pass} ███ S4████ ${missing.join('█')}`  
      : `BANTCC █████<${pass} ███ S4█`;  
  },  
  
  const hasQuote = Boolean(p.quotation_refs?.length || p.has_quotation || p.quotation_id);  
  return hasQuote ? null : '████████quotation_refs/has_quotation█';  
},  
  
{  
  from: 'S4', to: 'S5', hard: true, // 2026-08-30 ███ C █████review-gate/████████████████████  
  check: (p) => {  
    const approved = Boolean(  
      p.review_gate_decision === 'approved' ||  
      (p.decisions || []).some(d => d.scene === 'REVIEW_GATE' && d.disposition === 'approved')  
    );  
    // █████████████████████—review-gate ██ / ███ / ████████  
    // █████ §3.1 █████contract_facts/signed_at/contract_no/order_date ... █████████████████████  
    if (approved) return null;  
    const hasContractFacts = Boolean(p.contract_facts || p.signed_at || p.contract_no);  
    const orderDate = p.swas?.schedule?.order_date;  
    if (hasContractFacts || orderDate) return null;  
    const parts = [];  
    parts.push(`█ review-gate ██████████`);  
    parts.push(`████████swas.schedule.order_date█`);  
    return parts.join('█') + `—hard █████████████████████`;  
  },  
},  
  
{  
  from: 'S5', to: 'S6', hard: true, // 2026-08-30 ███ C █████+██████████  
  check: (p) => {  
    const signed = Boolean((p.contract_no && p.signed_at) || p.delivery_accepted_at);  
    const paid = Boolean(p.paid_at || p.payment_received);  
    if (signed && paid) return null;  
    const parts = [];  
    if (!signed) parts.push(`██████/███`);  
    if (!paid) parts.push(`██████`);  
    return `${parts.join('█')}—hard █████+████████████████S3→S4 BANTCC ███`;  
  },  
},  
  
};
```

```
// █████████S_ATTACHMENT_GATES █████████ tagged attachment████ hard ███  
function hasTaggedAttachment(p, tag) {  
  return Array.isArray(p.attachments) && p.attachments.some((a) => a.tag === tag);  
}
```



```
}  
return { exploded: offenders.length > 0, offenders };  
}  
  
// ===== FILE: src/action/resolver.js =====  
// src/action/resolver.js - ██████████agents.md █████R5 ████████  
// █ Action Registry ███████████████████████ NL→Page █████ / █ agent L1 █ / MCP █  
// █████ai-native-action-design █████resolver █ registry ████████ Action/SKILL/████  
import { listActions, listNamespaces } from './registry.js';  
  
// █ registry → ██████████  
export function resolveCapabilityManifest() {  
    const namespaces = listNamespaces();  
    const actions = listActions().map(a => ({  
        name: a.name,  
        namespace: a.namespace,  
        kind: a.kind,  
        agentTool: !!a.agentTool,  
        needsApproval: !!a.needsApproval,  
        force: !!a.force,  
        version: a.version || '1.0.0',  
        owner: a.owner || 'crm-native',  
    }));  
    return { namespaces, actions };  
}  
  
// █ agents.md █ markdown████ namespace ████████/force/needsApproval█  
export function formatManifest() {  
    const m = resolveCapabilityManifest();  
    const lines = ['# ███████████████████████ Action Registry█', ''];  
    lines.push('## Action██████████R1-R6█', '');  
    for (const ns of m.namespaces) {  
        const nsActions = m.actions.filter(a => a.namespace === ns);  
        lines.push(`### ${ns}`);  
        for (const a of nsActions) {  
            const kindMark = a.kind === 'write' ? '█' : '█';  
            const flags = [  
                a.agentTool ? 'agentTool' : '',  
                a.needsApproval ? 'needsApproval' : '',  
                a.force ? 'force' : '',  
            ].filter(Boolean).join(',');  
            lines.push('- `${a.name}${kindMark}`${flags} ` [`${flags}]` : '' - v`${a.version}` owner=${a.owner}`);  
        }  
        lines.push('');  
    }  
    return lines.join('\n');
```

```
import { registerAction, listActions } from './registry.js';
import { query } from '../db.js';
import { createParticle, updateParticle, queryParticles, getParticle, createEdge, queryNeighbors } from '../particles/particleRepo.js';
import { advanceStage } from '../particles/lifecycle.js';
import { ruleEngine } from '../ruleEngine.js';
import { requireDecision } from '../decision/autonomyEngine.js';
import { emit } from '../events/bus.js';
import { listMetaAttr, setMetaAttr } from '../metaAttr/metaAttrRepo.js';
import { readConfig } from '../config/configStore.js';
import { recordFailure } from '../monitor/monitorStore.js';
import {
    S_STAGES, S_LABEL, S_TRANSITIONS, S_GATE_DEFS, S_ATTACHMENT_GATES, toStageCode,
} from '../sales/stageTaxonomy.js';
import { scoreBantcc6 } from '../sales/gateThresholdDelta.js';
import { mantOk, funnelZone, forecastClass, weightedAmount } from '../sales/funnelQuality.js';
import { evaluateBehaviorChecklist } from '../sales/behaviorChecklist.js';
import { mergedThresholds, readThreshold, deriveRhythmDays } from '../sales/salesThresholds.js';
import { reopenDeal } from '../sales/reopenDeal.js';
import { mergedTargets } from '../sales/namedAccountTargets.js';

// #####2026-09-02#####crm-deal-advance ■ last_decision_id ■■ updateParticle ■■■
// ##### = #####/#####
//   handler ■■updated.payload = {...}` #####advanceStage ■■ stage■■
// #####S1→S2####need_facts 3 #####product/qty/spec■ → #####decision-gate.test.js:37-41■■

// #####2026-09-02#####agent ■■■ action ■ SQL #####
// #####→ ■■ relation does not exist■
// ##### catch ##### → #####SKILL #####
// #####catch ■■ recordFailure(kind, e) ##### — fail-safe #####
// ■■test/method-skill-real-execution.test.js #####
//   #####

// ■■ → ##### 0 #####
// 2026-08-31 ##### S1-S8 #####
// 2026-09-02 ##### S3 #####S3→QUOTE_PRICING / S4→SIGN_RISK / S5→POST_CONTRACT■■
//   #####S8 ##### OPP_QUALIFY■
//   #####decision ■ SOLUTION_VALUE / CLIENT_STRATEGY / POST_CONTRACT ##### 0 ■■
//   #####src/sales/stageTaxonomy.js S_LABEL|S3 ##### / S4 ##### / S5 ##### / S6 #####
const STAGE_SCENARIO = {
    S1: 'LEAD_FOLLOW_UP', S2: 'OPP_QUALIFY', S3: 'SOLUTION_VALUE',
    S4: 'QUOTE_PRICING', S5: 'SIGN_RISK', S6: 'POST_CONTRACT',
    S7: 'LOSS_REVIEW', S8: 'LOSS_REVIEW',
};

// ■■ A #####2026-08-31#####submit ■■ flow_id ■■ business_type → domain ##### id
//   ■■ flow_id ##### id ##### getFlowByDomain ■■
const BIZ_DOMAIN = {
    CRM_QUOTATION: 'quote', CRM_CONTRACT: 'contract', CRM_INVOICE: 'invoice',
    CRM_ORDER: 'order', CRM DEAL: 'deal',
};
```

```
// T7 T8 T9 T10 T11 T12 T13 T14 T15 T16 T17 T18 T19 T20 T21 T22 T23 T24 T25 T26 T27 T28 T29 T30 T31 T32 T33 T34 T35 T36 T37 T38 T39 T40 T41 T42 T43 T44 T45 T46 T47 T48 T49 T50 T51 T52 T53 T54 T55 T56 T57 T58 T59 T60 T61 T62 T63 T64 T65 T66 T67 T68 T69 T70 T71 T72 T73 T74 T75 T76 T77 T78 T79 T80 T81 T82 T83 T84 T85 T86 T87 T88 T89 T90 T91 T92 T93 T94 T95 T96 T97 T98 T99 T100 T101 T102 T103 T104 T105 T106 T107 T108 T109 T110 T111 T112 T113 T114 T115 T116 T117 T118 T119 T120 T121 T122 T123 T124 T125 T126 T127 T128 T129 T130 T131 T132 T133 T134 T135 T136 T137 T138 T139 T140 T141 T142 T143 T144 T145 T146 T147 T148 T149 T150 T151 T152 T153 T154 T155 T156 T157 T158 T159 T160 T161 T162 T163 T164 T165 T166 T167 T168 T169 T170 T171 T172 T173 T174 T175 T176 T177 T178 T179 T180 T181 T182 T183 T184 T185 T186 T187 T188 T189 T190 T191 T192 T193 T194 T195 T196 T197 T198 T199 T200 T201 T202 T203 T204 T205 T206 T207 T208 T209 T210 T211 T212 T213 T214 T215 T216 T217 T218 T219 T220 T221 T222 T223 T224 T225 T226 T227 T228 T229 T230 T231 T232 T233 T234 T235 T236 T237 T238 T239 T240 T241 T242 T243 T244 T245 T246 T247 T248 T249 T250 T251 T252 T253 T254 T255 T256 T257 T258 T259 T260 T261 T262 T263 T264 T265 T266 T267 T268 T269 T270 T271 T272 T273 T274 T275 T276 T277 T278 T279 T280 T281 T282 T283 T284 T285 T286 T287 T288 T289 T290 T291 T292 T293 T294 T295 T296 T297 T298 T299 T300 T301 T302 T303 T304 T305 T306 T307 T308 T309 T310 T311 T312 T313 T314 T315 T316 T317 T318 T319 T320 T321 T322 T323 T324 T325 T326 T327 T328 T329 T330 T331 T332 T333 T334 T335 T336 T337 T338 T339 T340 T341 T342 T343 T344 T345 T346 T347 T348 T349 T350 T351 T352 T353 T354 T355 T356 T357 T358 T359 T360 T361 T362 T363 T364 T365 T366 T367 T368 T369 T370 T371 T372 T373 T374 T375 T376 T377 T378 T379 T380 T381 T382 T383 T384 T385 T386 T387 T388 T389 T390 T391 T392 T393 T394 T395 T396 T397 T398 T399 T400 T401 T402 T403 T404 T405 T406 T407 T408 T409 T410 T411 T412 T413 T414 T415 T416 T417 T418 T419 T420 T421 T422 T423 T424 T425 T426 T427 T428 T429 T430 T431 T432 T433 T434 T435 T436 T437 T438 T439 T440 T441 T442 T443 T444 T445 T446 T447 T448 T449 T450 T451 T452 T453 T454 T455 T456 T457 T458 T459 T460 T461 T462 T463 T464 T465 T466 T467 T468 T469 T470 T471 T472 T473 T474 T475 T476 T477 T478 T479 T480 T481 T482 T483 T484 T485 T486 T487 T488 T489 T490 T491 T492 T493 T494 T495 T496 T497 T498 T499 T500 T501 T502 T503 T504 T505 T506 T507 T508 T509 T510 T511 T512 T513 T514 T515 T516 T517 T518 T519 T520 T521 T522 T523 T524 T525 T526 T527 T528 T529 T530 T531 T532 T533 T534 T535 T536 T537 T538 T539 T540 T541 T542 T543 T544 T545 T546 T547 T548 T549 T550 T551 T552 T553 T554 T555 T556 T557 T558 T559 T560 T561 T562 T563 T564 T565 T566 T567 T568 T569 T570 T571 T572 T573 T574 T575 T576 T577 T578 T579 T580 T581 T582 T583 T584 T585 T586 T587 T588 T589 T590 T591 T592 T593 T594 T595 T596 T597 T598 T599 T600 T601 T602 T603 T604 T605 T606 T607 T608 T609 T610 T611 T612 T613 T614 T615 T616 T617 T618 T619 T620 T621 T622 T623 T624 T625 T626 T627 T628 T629 T630 T631 T632 T633 T634 T635 T636 T637 T638 T639 T640 T641 T642 T643 T644 T645 T646 T647 T648 T649 T650 T651 T652 T653 T654 T655 T656 T657 T658 T659 T660 T661 T662 T663 T664 T665 T666 T667 T668 T669 T670 T671 T672 T673 T674 T675 T676 T677 T678 T679 T680 T681 T682 T683 T684 T685 T686 T687 T688 T689 T690 T691 T692 T693 T694 T695 T696 T697 T698 T699 T700 T701 T702 T703 T704 T705 T706 T707 T708 T709 T710 T711 T712 T713 T714 T715 T716 T717 T718 T719 T720 T721 T722 T723 T724 T725 T726 T727 T728 T729 T730 T731 T732 T733 T734 T735 T736 T737 T738 T739 T740 T741 T742 T743 T744 T745 T746 T747 T748 T749 T750 T751 T752 T753 T754 T755 T756 T757 T758 T759 T760 T761 T762 T763 T764 T765 T766 T767 T768 T769 T770 T771 T772 T773 T774 T775 T776 T777 T778 T779 T780 T781 T782 T783 T784 T785 T786 T787 T788 T789 T790 T791 T792 T793 T794 T795 T796 T797 T798 T799 T800 T801 T802 T803 T804 T805 T806 T807 T808 T809 T810 T811 T812 T813 T814 T815 T816 T817 T818 T819 T820 T821 T822 T823 T824 T825 T826 T827 T828 T829 T830 T831 T832 T833 T834 T835 T836 T837 T838 T839 T840 T841 T842 T843 T844 T845 T846 T847 T848 T849 T850 T851 T852 T853 T854 T855 T856 T857 T858 T859 T860 T861 T862 T863 T864 T865 T866 T867 T868 T869 T870 T871 T872 T873 T874 T875 T876 T877 T878 T879 T880 T881 T882 T883 T884 T885 T886 T887 T888 T889 T890 T891 T892 T893 T894 T895 T896 T897 T898 T899 T900 T901 T902 T903 T904 T905 T906 T907 T908 T909 T910 T911 T912 T913 T914 T915 T916 T917 T918 T919 T920 T921 T922 T923 T924 T925 T926 T927 T928 T929 T930 T931 T932 T933 T934 T935 T936 T937 T938 T939 T940 T941 T942 T943 T944 T945 T946 T947 T948 T949 T950 T951 T952 T953 T954 T955 T956 T957 T958 T959 T960 T961 T962 T963 T964 T965 T966 T967 T968 T969 T970 T971 T972 T973 T974 T975 T976 T977 T978 T979 T980 T981 T982 T983 T984 T985 T986 T987 T988 T989 T990 T991 T992 T993 T994 T995 T996 T997 T998 T999 1000 1001 1002 1003 1004 1005 1006 1007 1008 1009 1010 1011 1012 1013 1014 1015 1016 1017 1018 1019 1020 1021 1022 1023 1024 1025 1026 1027 1028 1029 1030 1031 1032 1033 1034 1035 1036 1037 1038 1039 1040 1041 1042 10
```


[illegible]

```
const passed = Object.values(dims).filter(Boolean).length;
const verdict = passed >= 3 ? 'pass' : passed === 2 ? 'conditional' : 'fail';
return { ok: true, deal_id: deal_id, dims, passedCount: passed, total: 4, verdict,
  findings: Object.entries(dims).filter(([v]) => !v).map(([k]) => `■: ${k}`) };
},
});
// crm-followup-schedule behavior-standard + 
registerAction({
  name: 'crm-followup-schedule', kind: 'read', permission: 'auth', requiresEntitlement: ['event_automation'],
  namespace: 'crm', agentTool: true, needsApproval: false,
  version: '1.0.0', owner: 'crm-native',
  schema: { deal_id: 'string' },
  parameters: { properties: { deal_id: { type: 'string', candidateSource: 'CRM_DEAL' } } },
  handler: async ({ deal_id }, ctx) => {
    const tid = ctx.tenantId || 'system';
    // 
    const cfg = (await readConfig('behavior-standard', { tenantId: tid }).catch(() => null))?.value || {};
    const cadenceDays = Number(cfg?.followup_cadence_days ?? 7);
    const rows = await query(
      `SELECT id, payload FROM crm.particles WHERE type='CRM_DEAL' AND tenant_id=$1
        ${deal_id ? 'AND id=$2' : ''} ORDER BY created_at DESC LIMIT 5`,
      [tid, deal_id]
    ).catch((e) => { recordFailure('crm-deal-lookup-failed', e); return { rows: [] }; });
    const now = Date.now();
    const schedule = (rows.rows || []).map((r) => {
      const p = r.payload || {};
      const last = p.last_activity_at ? new Date(p.last_activity_at).getTime() : (p.created_at ? new Date(p.created_at).getTime() : 0);
      const idleDays = Math.floor((now - last) / 864e5);
      const overdue = idleDays > cadenceDays;
      return { deal_id: r.id, idleDays, cadenceDays, overdue,
        action: overdue ? 'escalate_to_human' : 'auto_followup', nextAt: new Date(last + cadenceDays * 864e5).toISOString() };
    });
    return { ok: true, cadenceDays, schedule, overdueCount: schedule.filter((s) => s.overdue).length };
  },
});
// — CRM 2026-09-02 method-* —
// method-stage-progression / method-funnel-classification / method-behavior-standard
// seed.js steps[] executeSkill stepsMissing=true
// ① kind: 'read' ② HITL ③ src/sales/* 
// ③ readConfig ④ SQL 

// crm-stage-progression-evaluate S → + + BANTCC
registerAction({
  name: 'crm-stage-progression-evaluate', kind: 'read', permission: 'auth', requiresEntitlement: ['decision_autonomy'],
  namespace: 'crm', agentTool: true, needsApproval: false,
  version: '1.0.0', owner: 'crm-native',
  schema: { deal_id: 'string' },
  parameters: { properties: { deal_id: { type: 'string', candidateSource: 'CRM_DEAL' } } },
  handler: async ({ deal_id }, ctx) => {
```

[illegible]

[illegible]


```
-- 2026-09-05 fresh-install #####PK(423) ##### tenant_id##### ALTER #####
-- ##### schema.sql ##### → #####42703 column "tenant_id" does not exist#####
-- #####crm schema #####0##### CREATE ##### ADD COLUMN IF NOT EXISTS #####
tenant_id TEXT NOT NULL DEFAULT 'system',
title TEXT NOT NULL,
attr_type TEXT NOT NULL
CHECK (attr_type IN ('text','personal-name','email-address','phone-number','domain','location',
'number','currency','percent','date','timestamp','select','multi-select',
'boolean','rating','url','record-reference','actor-reference','interaction')),
semantic_tag TEXT NOT NULL DEFAULT 'legacy',
required BOOLEAN NOT NULL DEFAULT false,
"unique" BOOLEAN NOT NULL DEFAULT false,
description TEXT,
options JSONB,
source TEXT NOT NULL DEFAULT 'manual' CHECK (source IN ('manual','ai','enrich','automatic')),
display JSONB NOT NULL DEFAULT '{}':jsonb,
validation JSONB NOT NULL DEFAULT '{}':jsonb,
permission JSONB NOT NULL DEFAULT '{}':jsonb,
enabled BOOLEAN NOT NULL DEFAULT false,
version INTEGER NOT NULL DEFAULT 1,
created_by TEXT,
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
-- 2026-09-04 #####tenant_id ##### PK##### PK (particle_type, attr_slug)#####
-- ##### PK##### configCenter id19 ##### tenant #####
-- ##### migration-meta-attr-tenant-pk.sql #####
PRIMARY KEY (particle_type, attr_slug, tenant_id)
);
CREATE INDEX IF NOT EXISTS idx_crm_meta_attr_type_tag ON crm.meta_attr(particle_type, semantic_tag);

-- 2026-09-03 #####meta_attr ##### tenant_id#####
ALTER TABLE crm.meta_attr ADD COLUMN IF NOT EXISTS tenant_id TEXT NOT NULL DEFAULT 'system';
CREATE INDEX IF NOT EXISTS idx_crm_meta_attr_enabled ON crm.meta_attr(particle_type, enabled);

-- #####v2 #####Home.html → token → index.html#####
CREATE TABLE IF NOT EXISTS crm.crm_users (
user_id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
username text UNIQUE NOT NULL,
password_hash text NOT NULL,
role text NOT NULL,
display_name text NOT NULL,
org_id text,
enabled boolean NOT NULL DEFAULT true,
activated boolean NOT NULL DEFAULT true, -- #####DEFAULT true=##### false#####
tenant_id text NOT NULL DEFAULT 'system',
expires_at timestamptz, -- #####NULL=#####/#####
created_at timestamptz NOT NULL DEFAULT now()
);

-- ===== 2026-09-04#####→#####/#####→#####=====
```



```

updated_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE (doc_path, task, gap_type)
);
CREATE INDEX IF NOT EXISTS idx_contract_feedback_doc
  ON crm.contract_feedback(doc_path, task);

-- ===== living contract 2026-08-29 Task3 =====
-- contract_feedback doc_path,task,gap_type contract_task_id,gap_type
-- upsert/ON CONFLICT
CREATE TABLE IF NOT EXISTS crm.agent_contract_feedback (
  id BIGSERIAL PRIMARY KEY,
  contract_task_id text NOT NULL,
  agent text,
  gap_type text NOT NULL,
  observed text,
  expected text,
  severity text DEFAULT 'medium',
  ts timestamptz DEFAULT now(),
  resolved bool DEFAULT false,
  UNIQUE (contract_task_id, gap_type)
);

-- ===== 2026-08-30 =====
--  → LLM →
-- DELETE INSERT calibration_patch@
-- decision_retro_report calibration_patch calibration_patch
CREATE TABLE IF NOT EXISTS crm.decision_retro_report (
  report_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  run_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  window_start TIMESTAMPTZ NOT NULL,
  window_end TIMESTAMPTZ NOT NULL,
  decisions_scanned INT NOT NULL DEFAULT 0,
  clusters JSONB NOT NULL DEFAULT '[]'::jsonb, -- + +
  draft_patches JSONB NOT NULL DEFAULT '[]'::jsonb, --
  summary JSONB NOT NULL DEFAULT '{}'::jsonb, -- / / llm
  llm_enabled BOOLEAN NOT NULL DEFAULT false,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX IF NOT EXISTS idx_retro_report_run ON crm.decision_retro_report(run_at DESC);

-- R4 2026-09-05 decision.decided_at / tasks.updated_at Seq Scan
-- src/decision/retro.js loadWindowDecisions SELECT
CREATE INDEX IF NOT EXISTS idx_crm_decision_decided_at ON crm.decision (decided_at DESC);
CREATE INDEX IF NOT EXISTS idx_crm_tasks_updated_at ON crm.tasks (updated_at DESC);

-- ===== 2026-08-30 DDL =====
-- IF NOT EXISTS / ADD COLUMN IF NOT EXISTS / DROP+ADD CHECK
-- T2 decision_relation / T5 decision_outcome+outcome_event_map / T8 confidence+outcome_verified+feedback+root_cause
-- / T19+T28 calibration_patch.knob / T31 meta_attr.source_refresh_sla

```



```

CHECK (knob IN ('threshold','weight','required_dims','confidence','edge_binding',
               'outcome_threshold','strictness','meta_attr_map','particle_attr_add',
               'k_edge_add','source_refresh','dim_order','precedent_distill','config_store',
               'routing_tracks','routing_weight','routing_threshold')));
ALTER TABLE crm.calibration_patch ADD COLUMN IF NOT EXISTS assignee TEXT NOT NULL DEFAULT 'ADMIN';
ALTER TABLE crm.calibration_patch ADD COLUMN IF NOT EXISTS tenant_id TEXT NOT NULL DEFAULT 'system';
CREATE INDEX IF NOT EXISTS idx_calibration_patch_todo
  ON crm.calibration_patch(status, assignee, tenant_id, created_at DESC);

```

```
- G2 ■■ DB ■■T2 ■■■■■■decision_rule ■■■ + rule_hit ■■■
CREATE TABLE IF NOT EXISTS crm.decision_rule (
  id          BIGSERIAL PRIMARY KEY,
  code        TEXT UNIQUE NOT NULL,
  match_type  TEXT NOT NULL,          -- 'CRM_DEAL.advance'■■■■■■■■
  match_payload JSONB NOT NULL DEFAULT '{}':jsonb,
  check_payload JSONB NOT NULL DEFAULT '{}':jsonb,
  enabled      BOOLEAN NOT NULL DEFAULT true,
  decision_id  UUID REFERENCES crm.decision(decision_id), -- ■■■■0■■■
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX IF NOT EXISTS idx_crm_decision_rule_enabled ON crm.decision_rule(enabled);
```

```
CREATE TABLE IF NOT EXISTS crm.rule_hit (
  id          BIGSERIAL PRIMARY KEY,
  rule_code   TEXT NOT NULL,
  decision_id UUID,
  blocked     BOOLEAN NOT NULL,
  reasons     JSONB NOT NULL DEFAULT '[]'::jsonb,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX IF NOT EXISTS idx_crm_rule_hit_rule ON crm.rule_hit(rule_code);
CREATE INDEX IF NOT EXISTS idx_crm_rule_hit_blocked ON crm.rule_hit(blocked);
```

[illegible]


```

cycle         TEXT NOT NULL DEFAULT 'monthly' CHECK (cycle IN ('monthly','quarterly')),
token_in      INT NOT NULL DEFAULT 0,
token_out     INT NOT NULL DEFAULT 0,
token_fee     NUMERIC(12,2) NOT NULL DEFAULT 0,
seat_count    INT NOT NULL DEFAULT 0,
seat_fee      NUMERIC(12,2) NOT NULL DEFAULT 0,
total_fee     NUMERIC(12,2) NOT NULL DEFAULT 0,
status        TEXT NOT NULL DEFAULT 'draft' CHECK (status IN ('draft','issued','paid','overdue')),
issued_at     TIMESTAMPTZ,
due_at        TIMESTAMPTZ,
paid_at       TIMESTAMPTZ,
created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
              UNIQUE (tenant_id, period, cycle)
);

CREATE TABLE IF NOT EXISTS crm.billing_payment (
    id          BIGSERIAL PRIMARY KEY,
    tenant_id   TEXT NOT NULL,
    statement_id BIGINT NOT NULL REFERENCES crm.billing_statement(id),
    amount      NUMERIC(12,2) NOT NULL,
    method      TEXT NOT NULL CHECK (method IN ('bank_transfer','wechat','alipay','other')),
    status      TEXT NOT NULL DEFAULT 'paid' CHECK (status IN ('pending','paid','failed')),
    paid_at     TIMESTAMPTZ,
    txn_ref     TEXT,
    note        TEXT,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX IF NOT EXISTS idx_crm_billing_statement_tenant_period
ON crm.billing_statement(tenant_id, period, cycle);

CREATE INDEX IF NOT EXISTS idx_crm_billing_payment_statement
ON crm.billing_payment(statement_id);

-- ■■■append-only■■■=■■■+■■■■■■■■■■ DELETE■■■■■■■ tenant_id NOT NULL■
--      status ■■ CHECK■UNIQUE(tenant_id,plan_id,started_at) ■■■■■■
--      ■■■■pending■■■■■■→ active■■■■■■→ expired■■■■■■/ canceled■■■■■■/ grace■■■■■

CREATE TABLE IF NOT EXISTS crm.tenant_subscription (
    id          BIGSERIAL PRIMARY KEY,
    tenant_id   TEXT NOT NULL,
    plan_id     TEXT NOT NULL,
    status      TEXT NOT NULL DEFAULT 'active'
                CHECK (status IN ('active','pending','expired','canceled','grace')),
    started_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    expires_at  TIMESTAMPTZ NOT NULL,
    grace_until TIMESTAMPTZ,
    payment_ref BIGINT REFERENCES crm.billing_payment(id),
    online_order_no TEXT, -- ■■■■■■■■■■/■■■ out_trade_no■■■■■ payment_ref(■■■)
    upgraded_from TEXT,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE (tenant_id, plan_id, started_at)
);

```


[illegible]

```
-- ===== FILE: db/seed-master-data.sql =====
-- db/seed-master-data.sql - ██████████P0 ████
-- █████docs/superpowers/specs/2026-08-28-business-master-data-design.md
-- █████docs/superpowers/plans/2026-08-28-business-master-data-impl.md
--
-- ████████████████████**████ TRUNCATE / DELETE / DROP████ INSERT █████
-- `ON CONFLICT (id) DO NOTHING` → ████████████████████████████████████████
-- █████node scripts/seed-master-data.mjs
--
-- state ██████████ flow████████████████████PATCH /api/particles/:id██████████
-- CRM_PRODUCT → on_sale / discontinued
-- CRM_PRICE_LIST → draft / active / expired
-- CRM_OFFER_POLICY → draft / active / expired
-- CRM_DICT_ENTRY → registered / deprecated
-- ██████████ activeDictValues() █████ state='registered' █ payload.active!==false █████
--
-- ===== ① █████████CRM_PRODUCT████=====
INSERT INTO crm.particles (id, tenant_id, type, slug, title, state, payload, created_at, updated_at) VALUES
('b0000001-0000-0000-0000-000000000001', 'system', 'CRM_PRODUCT', 'product-crm-standard', 'CRM █████SaaS █████', 'on_sale',
{'name':"CRM █████SaaS █████", "unit":"█/█", "category":"████", "list_price":98000, "status":"on_sale"},
'2026-01-01T09:00:00+08:00', '2026-08-28T09:00:00+08:00'),
('b0000001-0000-0000-0000-000000000002', 'system', 'CRM_PRODUCT', 'product-colorbox', '████████', 'on_sale',
{'name':"████████", "unit":"","category":"████████", "list_price":45000, "status":"on_sale"},
'2026-01-01T09:00:00+08:00', '2026-08-28T09:00:00+08:00'),
('b0000001-0000-0000-0000-000000000003', 'system', 'CRM_PRODUCT', 'product-instruction', '████████', 'on_sale',
{'name':"████████", "unit":"","category":"████", "list_price":12000, "status":"on_sale"},
'2026-01-01T09:00:00+08:00', '2026-08-28T09:00:00+08:00'),
('b0000001-0000-0000-0000-000000000004', 'system', 'CRM_PRODUCT', 'product-implementation', '████', 'on_sale',
{'name':"████", "unit":"","category":"████", "list_price":2000, "status":"on_sale"},
'2026-01-01T09:00:00+08:00', '2026-08-28T09:00:00+08:00'),
('b0000001-0000-0000-0000-000000000005', 'system', 'CRM_PRODUCT', 'product-giftbox', '████████', 'on_sale',
{'name':"████████", "unit":"","category":"██████", "list_price":38000, "status":"on_sale"},
'2026-01-01T09:00:00+08:00', '2026-08-28T09:00:00+08:00'),
('b0000001-0000-0000-0000-000000000006', 'system', 'CRM_PRODUCT', 'product-carton-special', '████████/████', 'on_sale',
{'name':"████████/████", "unit":"","category":"████████", "list_price":52000, "status":"on_sale"},
'2026-01-01T09:00:00+08:00', '2026-08-28T09:00:00+08:00'),
('b0000001-0000-0000-0000-000000000007', 'system', 'CRM_PRODUCT', 'product-direct-mail', '████████', 'on_sale',
{'name':"████████", "unit":"","category":"██████", "list_price":9000, "status":"on_sale"},
'2026-01-01T09:00:00+08:00', '2026-08-28T09:00:00+08:00'),
('b0000001-0000-0000-0000-000000000008', 'system', 'CRM_PRODUCT', 'product-tech-support', '██████████████', 'on_sale',
{'name':"██████████████", "unit":"","category":"████", "list_price":28000, "status":"on_sale"},
'2026-01-01T09:00:00+08:00', '2026-08-28T09:00:00+08:00'),
('b0000001-0000-0000-0000-000000000009', 'system', 'CRM_PRODUCT', 'product-crm-professional', 'CRM █████SaaS █████', 'on_sale',
{'name':"CRM █████SaaS █████", "unit":"","category":"████", "list_price":158000, "status":"on_sale"},
'2026-01-01T09:00:00+08:00', '2026-08-28T09:00:00+08:00'),
('b0000001-0000-0000-0000-000000000010', 'system', 'CRM_PRODUCT', 'product-poster-brochure', '███/████', 'on_sale',
{'name':"███/████", "unit":"","category":"██████", "list_price":26000, "status":"on_sale"},
'2026-01-01T09:00:00+08:00', '2026-08-28T09:00:00+08:00'),
('b0000001-0000-0000-0000-000000000011', 'system', 'CRM_PRODUCT', 'product-crm-enterprise', 'CRM ████████████████', 'on_sale',
{'name':"CRM ████████████████", "unit":"","category":"████", "list_price":680000, "status":"on_sale"},
'2026-01-01T09:00:00+08:00', '2026-08-28T09:00:00+08:00')
```

```
-- '2026-01-01T09:00:00+08:00', '2026-08-28T09:00:00+08:00'),
('b0000001-0000-0000-0000-000000000012', 'system', 'CRM_PRODUCT', 'product-voucher-label', '■■■■/■■■■■■■■', 'on_sale',
{'name':"■■■■/■■■■■■■■", "unit": "■■■", "category": "■■■■■■■■", "list_price": 6500, "status": "on_sale"}',
'2026-01-01T09:00:00+08:00', '2026-08-28T09:00:00+08:00')
ON CONFLICT (id) DO NOTHING;

-- ===== ② ■■■■■■■■CRM_PRICE_LIST■=====
-- ■■■■■■■■quoteService/priceCalc■■■■■state=active ■■■■
INSERT INTO crm.particles (id, tenant_id, type, slug, title, state, payload, created_at, updated_at) VALUES
('b0000002-0000-0000-0000-000000000001', 'system', 'CRM_PRICE_LIST', 'pricelist-2026-standard', '2026 ■■■■■■■■', 'active',
{'name':"2026 ■■■■■■■■", "valid_from": "2026-01-01", "valid_to": "2026-12-31", "permission": "internal", "products": ["CRM ■■■■■■■■SaaS ■■■■", "2026-01-01T09:00:00+08:00", '2026-08-28T09:00:00+08:00']},
('b0000002-0000-0000-0000-000000000002', 'system', 'CRM_PRICE_LIST', 'pricelist-2026-ka', '2026 ■■■■■■■■', 'active',
{'name':"2026 ■■■■■■■■", "valid_from": "2026-01-01", "valid_to": "2026-12-31", "permission": "restricted", "products": ["■■■■■■■■", "■■■■■■■■", "2026-03-01T09:00:00+08:00", '2026-08-28T09:00:00+08:00']},
('b0000002-0000-0000-0000-000000000003', 'system', 'CRM_PRICE_LIST', 'pricelist-2026-ecommerce', '2026 ■■■■■■■■', 'active',
{'name':"2026 ■■■■■■■■", "valid_from": "2026-01-01", "valid_to": "2026-12-31", "permission": "public", "products": ["■■■■■■■■", "■■/■■■■", "■■■■■■■■", "2026-05-01T09:00:00+08:00", '2026-08-28T09:00:00+08:00']},
('b0000002-0000-0000-0000-000000000004', 'system', 'CRM_PRICE_LIST', 'pricelist-2026-renewal', '2026 ■■■■■■■■', 'draft',
{'name':"2026 ■■■■■■■■", "valid_from": "2026-07-01", "valid_to": "2027-06-30", "permission": "internal", "products": ["CRM ■■■■■■■■SaaS ■■■■", "2026-07-01T09:00:00+08:00", '2026-08-28T09:00:00+08:00']},
ON CONFLICT (id) DO NOTHING;

-- ===== ③ ■■■■■■■■CRM_OFFER_POLICY, subtype=standard■=====
-- cost_structure / price_bands / tier_discount ■ JSON ■■■■■■■■payload JSONB ■■■■■■■■
-- ■ offerPolicyRender ■ resolvePriceBands / costTotal / marginView ■■■■■■■■ + ■■■■■■■■
INSERT INTO crm.particles (id, tenant_id, type, slug, title, state, payload, created_at, updated_at) VALUES
('b0000003-0000-0000-0000-000000000001', 'system', 'CRM_OFFER_POLICY', 'policy-standard-2026', '■■■■■■■■■■+■■■■', 'active',
{'name':"■■■■■■■■■■+■■■■", "subtype": "standard", "cost_structure": "[{\\"item\\": \\"■■■■\\", \\"cost\\": 52000}, {\\"item\\": \\"■■■■\\", \\"cost\\": 100000}]", "valid_from": "2026-01-01T09:00:00+08:00", '2026-08-28T09:00:00+08:00'}},
('b0000003-0000-0000-0000-000000000002', 'system', 'CRM_OFFER_POLICY', 'policy-ka-2026', '■■■■■■■■■■■■■■■■■■■■', 'active',
{'name':"■■■■■■■■■■■■■■■■■■■■", "subtype": "standard", "cost_structure": "[{\\"item\\": \\"■■■■\\", \\"cost\\": 980000}, {\\"item\\": \\"■■■■\\", \\"cost\\": 1000000}]", "valid_from": "2026-01-01T09:00:00+08:00", '2026-08-28T09:00:00+08:00'}},
('b0000003-0000-0000-0000-000000000003', 'system', 'CRM_OFFER_POLICY', 'policy-pharma-traceability', '■■■■■■■■■■ + ■■■■■', 'active',
{'name':"■■■■■■■■■■ + ■■■■■", "subtype": "standard", "cost_structure": "[{\\"item\\": \\"■■■■\\", \\"cost\\": 8500}, {\\"item\\": \\"■■■■■■■■\\", \\"cost\\": 1000000}]", "valid_from": "2026-02-01T09:00:00+08:00", '2026-08-28T09:00:00+08:00'}},
('b0000003-0000-0000-0000-000000000004', 'system', 'CRM_OFFER_POLICY', 'policy-ecommerce-pack', '■■■■■■■■■■', 'active',
{'name':"■■■■■■■■■■", "subtype": "standard", "cost_structure": "[{\\"item\\": \\"■■■■\\", \\"cost\\": 28000}, {\\"item\\": \\"■■■■\\", \\"cost\\": 6000}]", "valid_from": "2026-03-01T09:00:00+08:00", '2026-08-28T09:00:00+08:00'}},
('b0000003-0000-0000-0000-000000000005', 'system', 'CRM_OFFER_POLICY', 'policy-preprint-point', '■■■■■■■■■■', 'draft',
{'name':"■■■■■■■■■■", "subtype": "standard", "cost_structure": "[{\\"item\\": \\"■■■\\", \\"cost\\": 1800}, {\\"item\\": \\"■■■■\\", \\"cost\\": 2500}]", "valid_from": "2026-01-01T09:00:00+08:00", '2026-08-28T09:00:00+08:00'}},
ON CONFLICT (id) DO NOTHING;

-- ===== ④ ■■■■■■■■CRM_OFFER_POLICY, subtype=payment■=====
-- ■■■■■■■■■■■■■■ subtype='payment' ■■■■■■■■ subtype ■■■■
INSERT INTO crm.particles (id, tenant_id, type, slug, title, state, payload, created_at, updated_at) VALUES
('b0000005-0000-0000-0000-000000000001', 'system', 'CRM_OFFER_POLICY', 'paypolicy-standard-30', '■■■■■■■■■■30 ■■■■', 'active',
{'name':"■■■■■■■■■■30 ■■■■", "subtype": "payment", "payment_term": 30, "collection_tier": "■■■", "prepay_ratio": 0, "valid_from": "2026-01-01T09:00:00+08:00", '2026-01-01T09:00:00+08:00', '2026-08-28T09:00:00+08:00'}},
('b0000005-0000-0000-0000-000000000002', 'system', 'CRM_OFFER_POLICY', 'paypolicy-standard-30', '■■■■■■■■■■30 ■■■■', 'active',
{'name':"■■■■■■■■■■30 ■■■■", "subtype": "payment", "payment_term": 30, "collection_tier": "■■■", "prepay_ratio": 0, "valid_from": "2026-01-01T09:00:00+08:00", '2026-01-01T09:00:00+08:00', '2026-08-28T09:00:00+08:00'}},
ON CONFLICT (id) DO NOTHING;
```

```

('b0000005-0000-0000-0000-000000000002', 'system', 'CRM_OFFER_POLICY', 'paypolicy-ka-60', '██████████60 ████ + 20% ████', 'active',
{'name':"██████████60 ████ + 20% ████", "subtype":"payment", "payment_term":60, "collection_tier":"███", "prepay_ratio":20, "valid_from":
'2026-01-01T09:00:00+08:00', '2026-08-28T09:00:00+08:00'}),
('b0000005-0000-0000-0000-000000000003', 'system', 'CRM_OFFER_POLICY', 'paypolicy-ecom-30', '██████████30 ████', 'active',
{'name':"██████████30 ████", "subtype":"payment", "payment_term":30, "collection_tier":"███", "prepay_ratio":0, "valid_from": "2026-01-01T09:00:00+08:00", '2026-08-28T09:00:00+08:00'}),
('b0000005-0000-0000-0000-000000000004', 'system', 'CRM_OFFER_POLICY', 'paypolicy-pharma-90', '██████████90 ████ + 10% ████', 'active',
{'name':"██████████90 ████ + 10% ████", "subtype":"payment", "payment_term":90, "collection_tier":"███", "prepay_ratio":10, "valid_from":
'2026-02-01T09:00:00+08:00', '2026-08-28T09:00:00+08:00'})
ON CONFLICT (id) DO NOTHING;

-- ===== ⑤ █████CRM_DICT_ENTRY=====
-- █ meta-attr █████ select █████activeDictValues█state='registered' █ active!==false█
INSERT INTO crm.particles (id, tenant_id, type, slug, title, state, payload, created_at, updated_at) VALUES
-- industry████
('b0000004-0000-0000-0000-000000000001', 'system', 'CRM_DICT_ENTRY', 'dict-industry-1', '████', 'registered',
{'dict_key':"industry", "dict_value":"████", "sort_order":1, "active":true}, '2026-08-28T09:00:00+08:00', '2026-08-28T09:00:00+08:00'),
('b0000004-0000-0000-0000-000000000002', 'system', 'CRM_DICT_ENTRY', 'dict-industry-2', '██', 'registered',
{'dict_key':"industry", "dict_value":"██", "sort_order":2, "active":true}, '2026-08-28T09:00:00+08:00', '2026-08-28T09:00:00+08:00'),
('b0000004-0000-0000-0000-000000000003', 'system', 'CRM_DICT_ENTRY', 'dict-industry-3', '██', 'registered',
{'dict_key':"industry", "dict_value":"██", "sort_order":3, "active":true}, '2026-08-28T09:00:00+08:00', '2026-08-28T09:00:00+08:00'),
('b0000004-0000-0000-0000-000000000004', 'system', 'CRM_DICT_ENTRY', 'dict-industry-4', '████', 'registered',
{'dict_key':"industry", "dict_value":"████", "sort_order":4, "active":true}, '2026-08-28T09:00:00+08:00', '2026-08-28T09:00:00+08:00'),
-- size████
('b0000004-0000-0000-0000-000000000005', 'system', 'CRM_DICT_ENTRY', 'dict-size-1', '████', 'registered',
{'dict_key':"size", "dict_value":"████", "sort_order":1, "active":true}, '2026-08-28T09:00:00+08:00', '2026-08-28T09:00:00+08:00'),
('b0000004-0000-0000-0000-000000000006', 'system', 'CRM_DICT_ENTRY', 'dict-size-2', '████', 'registered',
{'dict_key':"size", "dict_value":"████", "sort_order":2, "active":true}, '2026-08-28T09:00:00+08:00', '2026-08-28T09:00:00+08:00'),
('b0000004-0000-0000-0000-000000000007', 'system', 'CRM_DICT_ENTRY', 'dict-size-3', '████', 'registered',
{'dict_key':"size", "dict_value":"████", "sort_order":3, "active":true}, '2026-08-28T09:00:00+08:00', '2026-08-28T09:00:00+08:00'),
('b0000004-0000-0000-0000-000000000008', 'system', 'CRM_DICT_ENTRY', 'dict-size-4', '████', 'registered',
{'dict_key':"size", "dict_value":"████", "sort_order":4, "active":true}, '2026-08-28T09:00:00+08:00', '2026-08-28T09:00:00+08:00'),
-- region████
('b0000004-0000-0000-0000-000000000009', 'system', 'CRM_DICT_ENTRY', 'dict-region-1', '██', 'registered',
{'dict_key':"region", "dict_value":"██", "sort_order":1, "active":true}, '2026-08-28T09:00:00+08:00', '2026-08-28T09:00:00+08:00'),
('b0000004-0000-0000-0000-000000000010', 'system', 'CRM_DICT_ENTRY', 'dict-region-2', '██', 'registered',
{'dict_key':"region", "dict_value":"██", "sort_order":2, "active":true}, '2026-08-28T09:00:00+08:00', '2026-08-28T09:00:00+08:00'),
('b0000004-0000-0000-0000-000000000011', 'system', 'CRM_DICT_ENTRY', 'dict-region-3', '██', 'registered',
{'dict_key':"region", "dict_value":"██", "sort_order":3, "active":true}, '2026-08-28T09:00:00+08:00', '2026-08-28T09:00:00+08:00'),
('b0000004-0000-0000-0000-000000000012', 'system', 'CRM_DICT_ENTRY', 'dict-region-4', '██', 'registered',
{'dict_key':"region", "dict_value":"██", "sort_order":4, "active":true}, '2026-08-28T09:00:00+08:00', '2026-08-28T09:00:00+08:00'),
-- payment_method██████
('b0000004-0000-0000-0000-000000000013', 'system', 'CRM_DICT_ENTRY', 'dict-pay-1', '██', 'registered',
{'dict_key':"payment_method", "dict_value":"██", "sort_order":1, "active":true}, '2026-08-28T09:00:00+08:00', '2026-08-28T09:00:00+08:00'),
('b0000004-0000-0000-0000-000000000014', 'system', 'CRM_DICT_ENTRY', 'dict-pay-2', '████', 'registered',
{'dict_key':"payment_method", "dict_value":"████", "sort_order":2, "active":true}, '2026-08-28T09:00:00+08:00', '2026-08-28T09:00:00+08:00'),
('b0000004-0000-0000-0000-000000000015', 'system', 'CRM_DICT_ENTRY', 'dict-pay-3', '████', 'registered',
{'dict_key':"payment_method", "dict_value":"████", "sort_order":3, "active":true}, '2026-08-28T09:00:00+08:00', '2026-08-28T09:00:00+08:00'),
-- decision_power██████
('b0000004-0000-0000-0000-000000000016', 'system', 'CRM_DICT_ENTRY', 'dict-power-1', 'high', 'registered',

```

```

{"dict_key":"decision_power","dict_value":"high","sort_order":1,"active":true}', '2026-08-28T09:00:00+08:00', '2026-08-28T09:00:00+08:00',
('b0000004-0000-0000-0000-000000000017', 'system', 'CRM_DICT_ENTRY', 'dict-power-2', 'medium', 'registered',
{'dict_key':"decision_power","dict_value":"medium","sort_order":2,"active":true}', '2026-08-28T09:00:00+08:00', '2026-08-28T09:00:00+08:00',
('b0000004-0000-0000-0000-000000000018', 'system', 'CRM_DICT_ENTRY', 'dict-power-3', 'low', 'registered',
{'dict_key':"decision_power","dict_value":"low","sort_order":3,"active":true}', '2026-08-28T09:00:00+08:00', '2026-08-28T09:00:00+08:00',
-- contact_level ████████
('b0000004-0000-0000-0000-000000000019', 'system', 'CRM_DICT_ENTRY', 'dict-level-1', '████', 'registered',
{'dict_key':"contact_level","dict_value":"████","sort_order":1,"active":true}', '2026-08-28T09:00:00+08:00', '2026-08-28T09:00:00+08:00',
('b0000004-0000-0000-0000-000000000020', 'system', 'CRM_DICT_ENTRY', 'dict-level-2', '████', 'registered',
{'dict_key':"contact_level","dict_value":"████","sort_order":2,"active":true}', '2026-08-28T09:00:00+08:00', '2026-08-28T09:00:00+08:00',
('b0000004-0000-0000-0000-000000000021', 'system', 'CRM_DICT_ENTRY', 'dict-level-3', '████', 'registered',
{'dict_key':"contact_level","dict_value":"████","sort_order":3,"active":true}', '2026-08-28T09:00:00+08:00', '2026-08-28T09:00:00+08:00',
-- industry █████5-6█
('b0000004-0000-0000-0000-000000000022', 'system', 'CRM_DICT_ENTRY', 'dict-industry-5', '██████', 'registered',
{'dict_key':"industry","dict_value":"██████","sort_order":5,"active":true}', '2026-08-28T09:00:00+08:00', '2026-08-28T09:00:00+08:00',
('b0000004-0000-0000-0000-000000000023', 'system', 'CRM_DICT_ENTRY', 'dict-industry-6', '██████', 'registered',
{'dict_key':"industry","dict_value":"██████","sort_order":6,"active":true}', '2026-08-28T09:00:00+08:00', '2026-08-28T09:00:00+08:00',
-- region █████5-7█
('b0000004-0000-0000-0000-000000000024', 'system', 'CRM_DICT_ENTRY', 'dict-region-5', '███', 'registered',
{'dict_key':"region","dict_value":"███","sort_order":5,"active":true}', '2026-08-28T09:00:00+08:00', '2026-08-28T09:00:00+08:00'),
('b0000004-0000-0000-0000-000000000025', 'system', 'CRM_DICT_ENTRY', 'dict-region-6', '███', 'registered',
{'dict_key':"region","dict_value":"███","sort_order":6,"active":true}', '2026-08-28T09:00:00+08:00', '2026-08-28T09:00:00+08:00'),
('b0000004-0000-0000-0000-000000000026', 'system', 'CRM_DICT_ENTRY', 'dict-region-7', '███', 'registered',
{'dict_key':"region","dict_value":"███","sort_order":7,"active":true}', '2026-08-28T09:00:00+08:00', '2026-08-28T09:00:00+08:00'),
-- payment_method █████4-5█
('b0000004-0000-0000-0000-000000000027', 'system', 'CRM_DICT_ENTRY', 'dict-pay-4', '██████', 'registered',
{'dict_key':"payment_method","dict_value":"██████","sort_order":4,"active":true}', '2026-08-28T09:00:00+08:00', '2026-08-28T09:00:00+08:00'),
('b0000004-0000-0000-0000-000000000028', 'system', 'CRM_DICT_ENTRY', 'dict-pay-5', '██████', 'registered',
{'dict_key':"payment_method","dict_value":"██████","sort_order":5,"active":true}', '2026-08-28T09:00:00+08:00', '2026-08-28T09:00:00+08:00'),
ON CONFLICT (id) DO NOTHING;

```

```
// ===== FILE: db/seed-named-account-patch.sql =====
-- db/seed-named-account-patch.sql - ████████████████████████ UPDATE█
-- █ seed.sql █ CRM_ACCOUNT █ named_owner/tier/state████ named-accounts █████
SET search_path TO crm, public;

UPDATE crm.particles
SET payload = jsonb_strip_nulls(payload)
|| jsonb_build_object(
    'named_owner', 'alice',
    'named_tier', '██',
    'named_state', 'active',
    'visit_notes', jsonb_build_array(
        jsonb_build_object('at', '2026-08-28T10:00:00+08:00', 'type', 'visit', 'objective', '████', 'result', '██ Q4 █████', 'next',
        jsonb_build_object('at', '2026-08-29T14:00:00+08:00', 'type', 'call', 'objective', '████', 'result', '██████████', 'next',
    )
),
updated_at = now()
WHERE id = 'a1111111-1111-1111-1111-111111111101'
AND type = 'CRM_ACCOUNT';
```

```
// ===== FILE: db/seed-users.sql =====
-- db/seed-users.sql - ██████████admin ██████
-- ██████████WHERE NOT EXISTS ██████████crypt ██████████src/http/auth.js / src/mcp/auth.js ██████
-- enabled █ tenant_id █ schema.sql ████████████████████
SET search_path TO crm, public;

INSERT INTO crm.crm_users (username, password_hash, role, display_name, org_id, email)
SELECT 'admin', crypt('admin123', gen_salt('bf')), 'admin', '██████', 'system', 'admin@123.com'
WHERE NOT EXISTS (SELECT 1 FROM crm.crm_users WHERE username = 'admin');

INSERT INTO crm.crm_users (username, password_hash, role, display_name, org_id, email)
SELECT 'alice', crypt('secret123', gen_salt('bf')), 'sales', 'Alice ███', 'system', 'alice@system.local'
WHERE NOT EXISTS (SELECT 1 FROM crm.crm_users WHERE username = 'alice');

-- ██████████ email ████████████████████/████████
UPDATE crm.crm_users SET email = 'admin@123.com' WHERE username = 'admin' AND email IS NULL;
UPDATE crm.crm_users SET email = 'alice@system.local' WHERE username = 'alice' AND email IS NULL;

INSERT INTO crm.crm_users (username, password_hash, role, display_name, org_id, email)
SELECT 'sysadmin', crypt('sysadmin123', gen_salt('bf')), 'sysadmin', '██████', 'system', 'sysadmin@system.local'
WHERE NOT EXISTS (SELECT 1 FROM crm.crm_users WHERE username = 'sysadmin');

UPDATE crm.crm_users SET email = 'sysadmin@system.local' WHERE username = 'sysadmin' AND email IS NULL;
// ===== FILE: db/seed.sql =====
-- db/seed.sql - ████████████████████Task 10 ████
-- ██████████CRM-ai-native ██████████crm.particles / crm.edges██████████████████
-- ████npm run seed → node db/migrate.js --seed → pool.query(seedSql) ██████████
-- ██████████/█████ UUID + ON CONFLICT (id) DO NOTHING██████████████████████████
-- ████████████████████████████████████████
-- ████crm.particles.state ████████████████████ ACTIVE██████████ payload.stage█
-- ██████ runRiskScan ███ payload.stage_changed_at ███ ai.stuck_warning█>30██████

SET search_path TO crm, public;

-- ===== ████████████████████alice ███ / ███ / █████=====
INSERT INTO crm.particles (id, tenant_id, type, slug, title, state, payload, created_at, updated_at) VALUES
('a1111111-1111-1111-1111-111111111101', 'system', 'CRM_ACCOUNT', 'account-yintong',
'██████████████████', 'ACTIVE',
'{"name":"██████████████████","industry":"██████","region":"████","business_title":"██████████████████████████████","source":"██████","size":"██████████████████"}',
'2026-06-01T09:00:00+08:00', '2026-08-20T09:00:00+08:00')
ON CONFLICT (id) DO NOTHING;

-- ===== ███ =====
INSERT INTO crm.particles (id, tenant_id, type, slug, title, state, payload, created_at, updated_at) VALUES
('a2222222-2222-2222-2222-222222222201', 'system', 'CRM_CONTACT', 'contact-liwei',
'████', 'ACTIVE',
'{"name":"████","email":"liwe@yintong-print.com","phone":"13800001111","title":"██████","department":"████","decision_power":"high","retention":"██████████████████"}',
'2026-06-02T10:00:00+08:00', '2026-08-20T09:00:00+08:00')
ON CONFLICT (id) DO NOTHING;
```

[illegible]

```
INSERT INTO crm.particles (id, tenant_id, type, slug, title, state, payload, created_at, updated_at) VALUES
('f2222222-2222-2222-2222-222222222222', 'system', 'CRM_QUOTATION', 'quote-paid',
'■■■■■■■■', 'ACTIVE',
'{"name":"■■■■■■■■","deal_id":"d4444444-4444-4444-4444-444444444444","valid_until":"2026-09-30","amount":260000,"items":[{"product_id":
"2026-06-25T09:00:00+08:00", "2026-06-25T09:00:00+08:00"}]
ON CONFLICT (id) DO NOTHING;

-- ===== ■■■ =====
INSERT INTO crm.particles (id, tenant_id, type, slug, title, state, payload, created_at, updated_at) VALUES
('c1111111-1111-1111-1111-111111111111', 'system', 'CRM_CONTRACT', 'contract-contracted',
'■■■■■■■■■■', 'effective',
'{"contract_no":"HT-2026-001","deal_id":"d3333333-3333-3333-3333-333333333333","quotation_id":"f1111111-1111-1111-1111-111111111111",
"2026-08-01T09:00:00+08:00", "2026-08-01T09:00:00+08:00"}
ON CONFLICT (id) DO NOTHING;

INSERT INTO crm.particles (id, tenant_id, type, slug, title, state, payload, created_at, updated_at) VALUES
('c2222222-2222-2222-2222-222222222222', 'system', 'CRM_CONTRACT', 'contract-paid',
'■■■■■■■■', 'effective',
'{"contract_no":"HT-2026-002","deal_id":"d4444444-4444-4444-4444-444444444444","quotation_id":"f2222222-2222-2222-2222-222222222222",
"2026-07-01T09:00:00+08:00", "2026-07-01T09:00:00+08:00"}
ON CONFLICT (id) DO NOTHING;

-- ===== ■■■ =====
INSERT INTO crm.particles (id, tenant_id, type, slug, title, state, payload, created_at, updated_at) VALUES
('b2111111-1111-1111-1111-111111111111', 'system', 'CRM_ORDER', 'order-contracted',
'■■■■■■■■■■■■', 'shipped',
'{"order_no":"S0-2026-001","deal_id":"d3333333-3333-3333-3333-333333333333","contract_id":"c1111111-1111-1111-1111-111111111111", "amc
"2026-08-05T09:00:00+08:00", "2026-08-15T09:00:00+08:00"}
ON CONFLICT (id) DO NOTHING;

INSERT INTO crm.particles (id, tenant_id, type, slug, title, state, payload, created_at, updated_at) VALUES
('b2222222-2222-2222-2222-222222222222', 'system', 'CRM_ORDER', 'order-paid',
'■■■■■■■■■■', 'completed',
'{"order_no":"S0-2026-002","deal_id":"d4444444-4444-4444-4444-444444444444","contract_id":"c2222222-2222-2222-2222-222222222222", "amc
"2026-07-05T09:00:00+08:00", "2026-07-12T09:00:00+08:00"}
ON CONFLICT (id) DO NOTHING;

-- ===== ■■■■■ =====
INSERT INTO crm.particles (id, tenant_id, type, slug, title, state, payload, created_at, updated_at) VALUES
('91111111-1111-1111-1111-111111111111', 'system', 'CRM_PAYMENT_PLAN', 'plan-contracted-1',
'■■■■■■■■■■■■■■■■■■', 'pending',
'{"contract_id":"c1111111-1111-1111-1111-111111111111","plan_amount":500000,"plan_end":"2026-08-15","plan_status":"pending"}',
"2026-08-01T09:00:00+08:00", "2026-08-01T09:00:00+08:00")
ON CONFLICT (id) DO NOTHING;

INSERT INTO crm.particles (id, tenant_id, type, slug, title, state, payload, created_at, updated_at) VALUES
('92222222-2222-2222-2222-222222222222', 'system', 'CRM_PAYMENT_PLAN', 'plan-paid-1',
'■■■■■■■■■■■■', 'done',
'{"contract_id":"c2222222-2222-2222-2222-222222222222","plan_amount":260000,"plan_end":"2026-07-20","plan_status":"done"}',
"2026-07-01T09:00:00+08:00", "2026-07-20T09:00:00+08:00")
```



```

-- ■■■ → ■■ / ■■■■■■
INSERT INTO crm.edges (id, tenant_id, source_type, source_id, edge_type, target_type, target_id, meta, created_at) VALUES
('e5555555-5555-5555-5555-555555555551', 'system', 'CRM_CONTRACT', 'c1111111-1111-1111-1111-111111111111', 'referenced_in', 'CRM_DEAL',
ON CONFLICT (id) DO NOTHING;
INSERT INTO crm.edges (id, tenant_id, source_type, source_id, edge_type, target_type, target_id, meta, created_at) VALUES
('e5555555-5555-5555-5555-555555555552', 'system', 'CRM_CONTRACT', 'c1111111-1111-1111-1111-111111111111', 'referenced_in', 'CRM_QUOTA',
ON CONFLICT (id) DO NOTHING;
INSERT INTO crm.edges (id, tenant_id, source_type, source_id, edge_type, target_type, target_id, meta, created_at) VALUES
('e5555555-5555-5555-5555-555555555553', 'system', 'CRM_CONTRACT', 'c2222222-2222-2222-2222-222222222222', 'referenced_in', 'CRM_DEAL',
ON CONFLICT (id) DO NOTHING;
INSERT INTO crm.edges (id, tenant_id, source_type, source_id, edge_type, target_type, target_id, meta, created_at) VALUES
('e5555555-5555-5555-5555-555555555554', 'system', 'CRM_CONTRACT', 'c2222222-2222-2222-2222-222222222222', 'referenced_in', 'CRM_QUOTA',
ON CONFLICT (id) DO NOTHING;

-- ■■■ → ■■ / ■■■■■■
INSERT INTO crm.edges (id, tenant_id, source_type, source_id, edge_type, target_type, target_id, meta, created_at) VALUES
('e6666666-6666-6666-6666-666666666661', 'system', 'CRM_ORDER', 'b2111111-1111-1111-1111-111111111111', 'referenced_in', 'CRM_CONTRACT',
ON CONFLICT (id) DO NOTHING;
INSERT INTO crm.edges (id, tenant_id, source_type, source_id, edge_type, target_type, target_id, meta, created_at) VALUES
('e6666666-6666-6666-6666-666666666662', 'system', 'CRM_ORDER', 'b2111111-1111-1111-1111-111111111111', 'referenced_in', 'CRM_DEAL',
ON CONFLICT (id) DO NOTHING;
INSERT INTO crm.edges (id, tenant_id, source_type, source_id, edge_type, target_type, target_id, meta, created_at) VALUES
('e6666666-6666-6666-6666-666666666663', 'system', 'CRM_ORDER', 'b2222222-2222-2222-2222-222222222222', 'referenced_in', 'CRM_CONTRACT',
ON CONFLICT (id) DO NOTHING;
INSERT INTO crm.edges (id, tenant_id, source_type, source_id, edge_type, target_type, target_id, meta, created_at) VALUES
('e6666666-6666-6666-6666-666666666664', 'system', 'CRM_ORDER', 'b2222222-2222-2222-2222-222222222222', 'referenced_in', 'CRM_DEAL',
ON CONFLICT (id) DO NOTHING;

-- ■■■■ → ■■■■■■
INSERT INTO crm.edges (id, tenant_id, source_type, source_id, edge_type, target_type, target_id, meta, created_at) VALUES
('e7777777-7777-7777-7777-777777777771', 'system', 'CRM_PAYMENT_PLAN', '91111111-1111-1111-1111-111111111111', 'referenced_in', 'CRM_C',
ON CONFLICT (id) DO NOTHING;
INSERT INTO crm.edges (id, tenant_id, source_type, source_id, edge_type, target_type, target_id, meta, created_at) VALUES
('e7777777-7777-7777-7777-777777777772', 'system', 'CRM_PAYMENT_PLAN', '92222222-2222-2222-2222-222222222222', 'referenced_in', 'CRM_C',
ON CONFLICT (id) DO NOTHING;

-- ■■■■ → ■■■■■■
INSERT INTO crm.edges (id, tenant_id, source_type, source_id, edge_type, target_type, target_id, meta, created_at) VALUES
('e8888888-8888-8888-8888-888888888881', 'system', 'CRM_PAYMENT_RECORD', '93333333-3333-3333-3333-333333333333', 'referenced_in', 'CRM',
ON CONFLICT (id) DO NOTHING;

-- ■■ → ■■■■■■
INSERT INTO crm.edges (id, tenant_id, source_type, source_id, edge_type, target_type, target_id, meta, created_at) VALUES
('e9999999-9999-9999-9999-999999999991', 'system', 'CRM_INVOICE', '94444444-4444-4444-4444-444444444444', 'referenced_in', 'CRM_CONTRA',
ON CONFLICT (id) DO NOTHING;

-- ===== ■■■■■■$6 ■■■■■■■■■■ plm 8 ■■■■■■■■■■/■■■=====
-- ■■■■■■docs/specs/2026-08-25-ai-native-crm-overall-design.md $6.5 ■■■■■■■■■■8 ■■■■
-- ■ db/schema.sql crm.decision_scenario ■■■■■■PK=(scenario_id, tenant_id)■2026-09-05 G5 ■■■■■■

```


[illegible]

[illegible]

```
-- item 215 severity/target_role NULL medium/ops
INSERT INTO crm.alert_rule (kind, match, check_params, enabled, version) VALUES
('deal_stuck', '{"particleTypes":["CRM_DEAL"],"actions":["stage_update","advance"]}':jsonb, '{"stuck_days":30}':jsonb, TRUE, 1),
('lead_overdue', '{"particleTypes":["CRM_DEAL"],"actions":["lead-picked","lead-recycled","followup","update"]}':jsonb, '{"overdue_c
```

```
-- ('forecast_breach', '{"particleTypes":["CRM_DEAL"],"actions":{"forecast_update","amount_update"}}':jsonb, '{"breach_pct":0.8}':jsonb,
('approval_bottleneck', '{"particleTypes":["**"],"actions":{"approval_create"}}':jsonb, '{"bottleneck_count":5}':jsonb, TRUE, 1),
('payment_due', '{"particleTypes":["CRM_INVOICE"],"actions":{"invoice_create"},"payment_create"}':jsonb, '{"due_days":7}':jsonb, T
-- 2026-09-06 alert_rule [REDACTED] (kind, tenant_id) [REDACTED] ON CONFLICT [REDACTED]
-- [REDACTED] seed.sql [REDACTED] no unique or exclusion constraint [REDACTED] reseedBase() [REDACTED]
ON CONFLICT (kind, tenant_id) DO UPDATE SET
match=EXCLUDED.match, check_params=EXCLUDED.check_params, enabled=EXCLUDED.enabled, updated_at=now();

-- ===== [REDACTED].=====
-- status [REDACTED] 'TODO' [REDACTED] engine.advanceTask [REDACTED] / [REDACTED]
-- [REDACTED]
-- approver [REDACTED] role:admin [REDACTED] admin [REDACTED] + role:sales [REDACTED] alice [REDACTED]
-- [REDACTED] UUID + ON CONFLICT DO NOTHING [REDACTED]
INSERT INTO crm.particles (id, tenant_id, type, slug, title, state, payload, created_at, updated_at) VALUES
('a9000000-0000-0000-0000-000000000001', 'system', 'CRM_APPROVAL_INSTANCE', 'demo-inst-1',
[REDACTED], 'ACTIVE',
['{"flow_id":"demo-flow","business_type":"CRM_QUOTATION","business_id":"f1111111-1111-1111-1111-111111111111","submitter":"alice","s
now(), now()),
('a9000000-0000-0000-0000-000000000011', 'system', 'CRM_APPROVAL_TASK', 'demo-task-admin',
[REDACTED], 'ACTIVE',
['{"instance_id":"a9000000-0000-0000-0000-000000000001","node_id":"n1","approver":"role:admin","status":"TODO","opinion":null,"seq":
now(), now()),
('a9000000-0000-0000-0000-000000000012', 'system', 'CRM_APPROVAL_TASK', 'demo-task-sales',
[REDACTED], 'ACTIVE',
['{"instance_id":"a9000000-0000-0000-0000-000000000001","node_id":"n1","approver":"role:sales","status":"TODO","opinion":null,"seq":
now(), now())
ON CONFLICT (id) DO NOTHING;

-- ===== [REDACTED] /agent-workbench [REDACTED] X [REDACTED] =====
-- [REDACTED] DEFAULT_CONTRACT_DOC = docs/specs/2026-08-29-agent-workbench-contract-monitor-design.md $3-$7
-- [REDACTED] 5 [REDACTED] contract task [REDACTED] task [REDACTED] 1 [REDACTED] loop-started + 1 [REDACTED] loop-done + 1 [REDACTED] context-injected [REDACTED]
-- [REDACTED] WHERE NOT EXISTS [REDACTED] monitor_event [REDACTED] UNIQUE [REDACTED]
INSERT INTO crm.monitor_event (domain, event_type, agent_id, context_facts, payload)
SELECT v.domain, v.event_type, v.agent_id, v.context_facts::jsonb, v.payload::jsonb
FROM (VALUES
-- A [REDACTED] intake-router [REDACTED] L1+L2 [REDACTED] skills: data-particle-read + method-intake-routing [REDACTED]
('agent','loop-started','intake-router',{'contract_task_id':"A [REDACTED] + [REDACTED] + [REDACTED]","skill":"data-particle-read"},
('agent','loop-done','intake-router',{'contract_task_id':"A [REDACTED] + [REDACTED] + [REDACTED]","skill":"method-intake-routing"},
('agent','context-injected','intake-router',{'contract_task_id':"A [REDACTED] + [REDACTED] + [REDACTED]","knowledge_layers_read":["L1",
-- B [REDACTED] quote-engine [REDACTED] L1+L2 [REDACTED] skills: data-particle-read + method-quote-engine [REDACTED]
('agent','loop-started','quote-engine',{'contract_task_id':"B [REDACTED]/[REDACTED]/[REDACTED] A/B [REDACTED]","skill":"data-particle-read"},{'})
('agent','loop-done','quote-engine',{'contract_task_id':"B [REDACTED]/[REDACTED]/[REDACTED] A/B [REDACTED]","skill":"method-quote-engine"},{'})
('agent','context-injected','quote-engine',{'contract_task_id':"B [REDACTED]/[REDACTED]/[REDACTED] A/B [REDACTED]","knowledge_layers_read":["L1",
-- C [REDACTED] followup-agent [REDACTED] L1+L2 [REDACTED] skills: data-particle-read + data-particle-create + method-followup-engine [REDACTED]
('agent','loop-started','followup-agent',{'contract_task_id':"C [REDACTED]/[REDACTED]/[REDACTED]","skill":"data-particle-read"},{'}),
('agent','loop-done','followup-agent',{'contract_task_id':"C [REDACTED]/[REDACTED]/[REDACTED]","skill":"data-particle-create"},{'}),
('agent','context-injected','followup-agent',{'contract_task_id':"C [REDACTED]/[REDACTED]/[REDACTED]","skill":"method-followup-engine"},{'}),
-- D [REDACTED] review-gate [REDACTED] L1+L2+L3 [REDACTED] skills: data-particle-read + method-review-gate [REDACTED]
('agent','loop-started','review-gate',{'contract_task_id':"D [REDACTED] + [REDACTED] + [REDACTED]","skill":"data-particle-read"},{'}),
```

```

('agent','loop-done','review-gate',{'contract_task_id':"D " + " " + " ","skill":"method-review-gate"},"{}'),
('agent','context-injected','review-gate',{'contract_task_id':"D " + " " + " ","knowledge_layers_read":["L1","L2"],
) AS v(domain, event_type, agent_id, context_facts, payload)
WHERE NOT EXISTS (
SELECT 1 FROM crm.monitor_event me
WHERE me.domain = v.domain
AND me.event_type = v.event_type
AND me.context_facts-->'contract_task_id' = v.context_facts::jsonb-->'contract_task_id'
AND me.context_facts-->'skill' IS NOT DISTINCT FROM v.context_facts::jsonb-->'skill'
);

-- " success "T4 " = pass" ✓ "
INSERT INTO crm.agent_contract_feedback (contract_task_id, agent, gap_type, observed, expected, severity)
VALUES
('D " + " " + " ', 'review-gate', 'success', 'pass',
' " ✓ ', 'info')
ON CONFLICT (contract_task_id, gap_type) DO UPDATE SET
observed = EXCLUDED.observed, ts = now();

// ===== FILE: db/test-setup.sql =====
-- db/test-setup.sql - " crm "
-- " /CI " PG" E2E " beforeEach TRUNCATE"/
TRUNCATE particles, edges, tasks, task_audit, scheduler_lock, events,
decision, decision_precedent_rel, decision_event, memory_log,
memory_snapshot, memory_note,
decision_scenario, methodology_template, methodology_dimension, policy_version, business_tier_config,
role_context_profile
RESTART IDENTITY CASCADE;

-- " $6 " db/seed.sql "
-- " TRUNCATE " decision_scenario" createDecision " decision_scenario_id_fkey " 2026-08-26 "
INSERT INTO crm.decision_scenario
(scenario_id, stage, description, trigger, methodology_ids, eval_dimensions, default_tier, autonomous_allowed) VALUES
('LEAD_FOLLOW_UP', ' ', ' / / / ',
{'cond':{'event':"created","stage":"lead"},"entity":"DEAL","source":"particle_event"}::jsonb,
ARRAY['BANT','MEDDIC','OPP_MATRIX'],
[{'cond':"industry_fit","label":" ","weight":0.2},{cond':"budget_cycle","label":" ", "weight":0.2},{cond':"pain_clear","label":"LEAD", TRUE),
('OPP_QUALIFY', ' ', ' / / / ',
{'cond':{'event':"qualify","stage":"opportunity"},"entity":"DEAL","source":"particle_event"}::jsonb,
ARRAY['MEDDIC','OPP_MATRIX','ROLE_MAP'],
[{'cond':"pain_source","label":" ", "weight":0.2},{cond':"budget_approved","label":" ", "weight":0.2},{cond':"decision_chain","label":"NORMAL", FALSE),
('CLIENT_STRATEGY', ' ', ' / - - - ',
{'cond':{'event':"client_strategy_due","stage":"opportunity"},"entity":"DEAL","source":"particle_event"}::jsonb,
ARRAY['ROLE_MAP','FACT_VS_TALK','MEDDIC'],
[{'cond':"role_identified","label":" ", "weight":0.2},{cond':"decision_chain","label":" ", "weight":0.2},{cond':"support","label":"NORMAL", FALSE),
('SOLUTION_VALUE', ' ', ' / / / ',
{'cond':{'event':"solution_due","stage":"'quoted'"},"entity":"DEAL","source":"particle_event"}::jsonb,

```


[illegible]

[illegible]

```
ON CONFLICT (id) DO NOTHING;
```

```
-- ■■■■■■■■■■■■item 21■5 ■■severity/target_role ■ NULL ■■■■■ medium/ops■
```

```
INSERT INTO crm.alert_rule (kind, match, check_params, enabled, version) VALUES
```

```
('deal_stuck', '{"particleTypes":["CRM_DEAL"],"actions":["stage_update","advance"]}::jsonb, {"stuck_days":30}::jsonb, TRUE, 1),
('lead_overdue', '{"particleTypes":["CRM_DEAL"],"actions":["lead-picked","lead-recycled","followup","update"]}::jsonb, {"overdue_d
('forecast_breach', '{"particleTypes":["CRM_DEAL"],"actions":["forecast_update","amount_update"]}::jsonb, {"breach_pct":0.8}::jso
('approval_bottleneck', '{"particleTypes":["*"],"actions":["approval_create"]}::jsonb, {"bottleneck_count":5}::jsonb, TRUE, 1),
('payment_due', '{"particleTypes":["CRM_INVOICE"],"actions":["invoice_create","payment_create"]}::jsonb, {"due_days":7}::jsonb, T
```

```
-- 2026-09-06 alert_rule (kind, tenant_id) ON CONFLICT test-setup.sql no unique or exclusion constraint
```

```
ON CONFLICT (kind, tenant_id) DO UPDATE SET
```

```
match=EXCLUDED.match, check_params=EXCLUDED.check_params, enabled=EXCLUDED.enabled, updated_at=now());
```